Module 2 — HTTP & Express (Status Codes & Headers)

Time: 1.5 hours · Covers **Practical 3** · **CO1**

> *Practical 3: Demonstrate how to send various HTTP status codes and response headers in Node.js.*

Part A — How the web talks: HTTP (concept, ~15 min)

Every web interaction is a **request** from a client (browser, mobile app) and a **response** from a server. This conversation follows the **HTTP** protocol.

A request has:

- a **method** — the verb: `GET` (read), `POST` (create), `PUT` / `PATCH` (update), `DELETE` (remove).
- a **URL / path** — what resource, e.g. `/users/1`.
- **headers** — metadata (content type, auth token, etc.).
- an optional **body** — data sent with `POST` / `PUT`.

A response has:

- a **status code** — a 3-digit number saying how it went.
- **headers** — metadata about the response.
- a **body** — the actual data (HTML, JSON, ...).

Part B — HTTP status codes (~20 min)

Status codes are grouped by their first digit:

Range	Meaning	Common examples
1xx	Informational	(rarely used directly)
2xx	**Success**	`200 OK`, `201 Created`, `204 No Content`
3xx	Redirection	`301 Moved Permanently`, `304 Not Modified`
4xx	**Client error**	`400 Bad Request`, `401 Unauthorized`, `403 Forbidden`, `404 Not Found`
5xx	**Server error**	`500 Internal Server Error`, `503 Service Unavailable`

Rules of thumb:

- Return `201` (not `200`) when you **create** a resource.
- Return `400` when the **client** sent bad data; `500` when **your server** crashed.
- Return `404` when the requested resource does not exist.

Part C — First, the raw `http` module (~15 min)

Before frameworks, this is what Node gives you out of the box. No install.

See `code/raw-http-server.js`. It shows how to set a status code and headers manually. Run it:

```
node code/raw-http-server.js
# then in another terminal:
curl -i http://localhost:3000/
```

The `-i` flag makes `curl` print the status line and headers, so you can *see* what the server sent.

Doing everything by hand is tedious. That is why we use **Express**.

Part D — Express (~30 min)

Express is the most popular Node web framework. It gives you clean routing and easy control over status codes, headers, and JSON.

Install (already in this module's `package.json`):

```
cd 02-http-express
npm install
node code/express-server.js
```

Sending a status code

```
// res.status(code) sets the code; .json(...) sends a JSON body.
app.get("/ok", (req, res) => {
  res.status(200).json({ message: "All good" });
});

app.post("/users", (req, res) => {
  // ...create the user...
  res.status(201).json({ id: 1, created: true }); // 201 = Created
});
```

Sending response headers

```
app.get("/with-headers", (req, res) => {
  res.set("X-Powered-By", "AdvancedWebCourse");   // one header
  res.set({                                       // many at once
    "Cache-Control": "no-store",
    "X-Request-Id": "abc-123",
  });
  res.status(200).json({ ok: true });
});
```

Handling errors with the right code

```
app.get("/users/:id", (req, res) => {
  const id = Number(req.params.id);
  if (Number.isNaN(id)) {
```

```
    return res.status(400).json({ error: "id must be a number" }); // client's fault
  }
  const user = findUser(id);
  if (!user) {
    return res.status(404).json({ error: "user not found" });      // does not exist
  }
  res.status(200).json(user);
});
```

The full, commented server lives in `code/express-server.js`. It has one route for **each** important status code so you can hit them with `curl` and watch the codes and headers change.

Part E — Try every route (~10 min)

With the server running:

```
curl -i http://localhost:3000/success        # 200
curl -i -X POST http://localhost:3000/users  # 201
curl -i http://localhost:3000/no-content     # 204 (no body)
curl -i http://localhost:3000/bad-request    # 400
curl -i http://localhost:3000/unauthorized   # 401
curl -i http://localhost:3000/not-found      # 404
curl -i http://localhost:3000/server-error   # 500
curl -i http://localhost:3000/headers-demo   # 200 + custom headers
```

Watch the first line of each response (`HTTP/1.1 <code> <text>`) and the headers below it.

Summary

- HTTP = request (method, path, headers, body) + response (status, headers, body).
- Status codes: 2xx success, 4xx client error, 5xx server error.
- In Express: `res.status(code)` , `res.set(header, value)` , `res.json(data)` .
- Pick the **correct** code — it is part of a good API's contract.

Now do `practice.md` .